MeatLens

User and Technical Manual

Version 1.0 | May 19, 2026

AI-Powered Wet Market Meat Freshness Inspection Platform

System Name	MeatLens
Prepared For	Inspectors, Administrators, and Technical Maintainers
Repository	botchabuster
Revision Date	May 19, 2026

This manual combines end-user guidance and technical operations details.

Table of Contents

1. Introduction

MeatLens is a role-based inspection platform for wet markets that combines camera capture, MobileNetV3-based image inference, confidence-aware recommendations, secure storage, and audit-ready reporting.

1.1 Intended Audience

- Field inspectors performing daily freshness inspections.
- Supervisors and administrators managing users, locations, and reports.
- Technical maintainers operating infrastructure, deployments, and integrations.

1.2 Scope of This Manual

- User workflow from account registration through inspection and reporting.
- Admin workflow for operational monitoring and account governance.
- Technical reference for architecture, APIs, data model, security, and deployment.

2. Getting Started

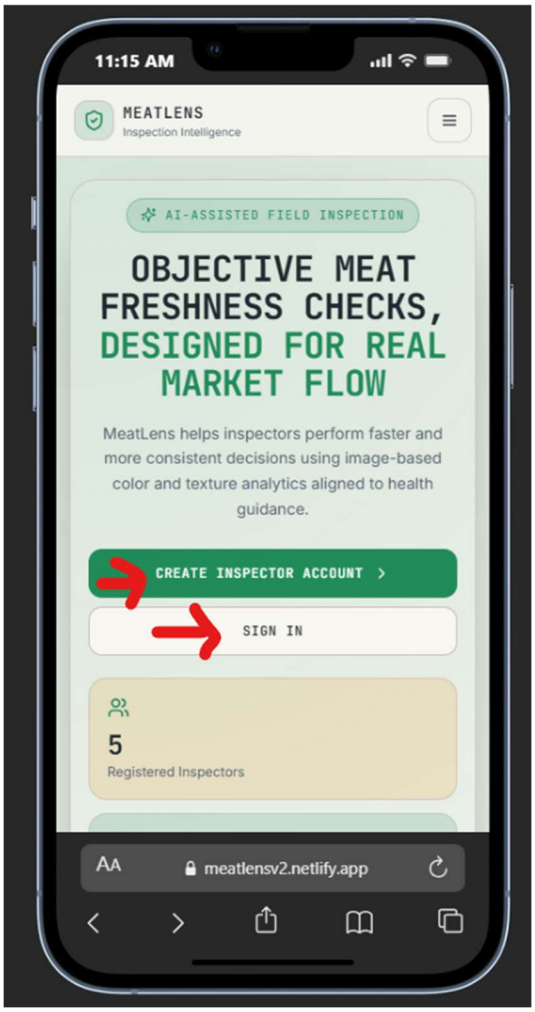
MeatLens is accessed through a modern web browser and is optimized for mobile-first field operations.

2.1 System Requirements

- Supported browsers: Chrome, Edge, Firefox, or Safari (current versions).
- Camera-enabled phone, tablet, or desktop with webcam.
- Stable network connection for cloud sync (offline capture is supported).
- Inspector account credentials and valid access code for registration.

2.2 Launching the Application

- Open the deployed MeatLens URL in your browser. https://meatlensv2.netlify.app/
- Choose `Sign In` for existing users or `Sign Up` for new users.
- After login, admins are routed to `/admin`; inspectors are routed to `/inspect`.



3. Authentication and Account Access

Authentication is handled via backend auth endpoints with Supabase-managed users.

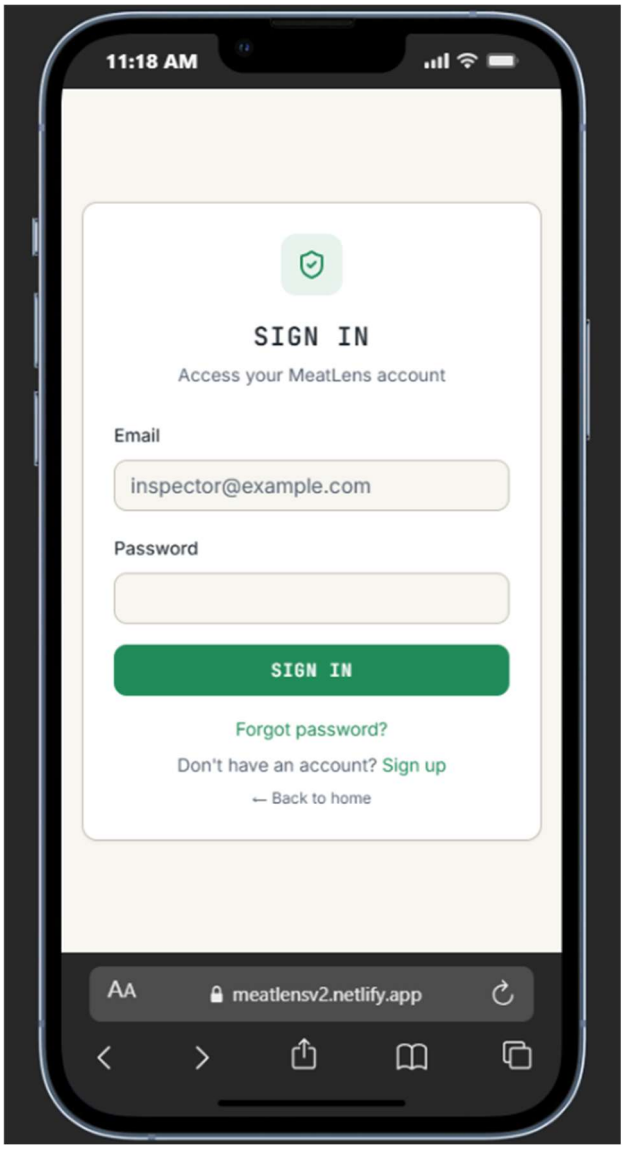
3.1 Sign Up (New Inspector)

- Enter full name, email, password (minimum 6 characters), and access code.
- Accept Terms and Conditions and Privacy Policy checkboxes.
- Submit account creation and verify email if prompted.

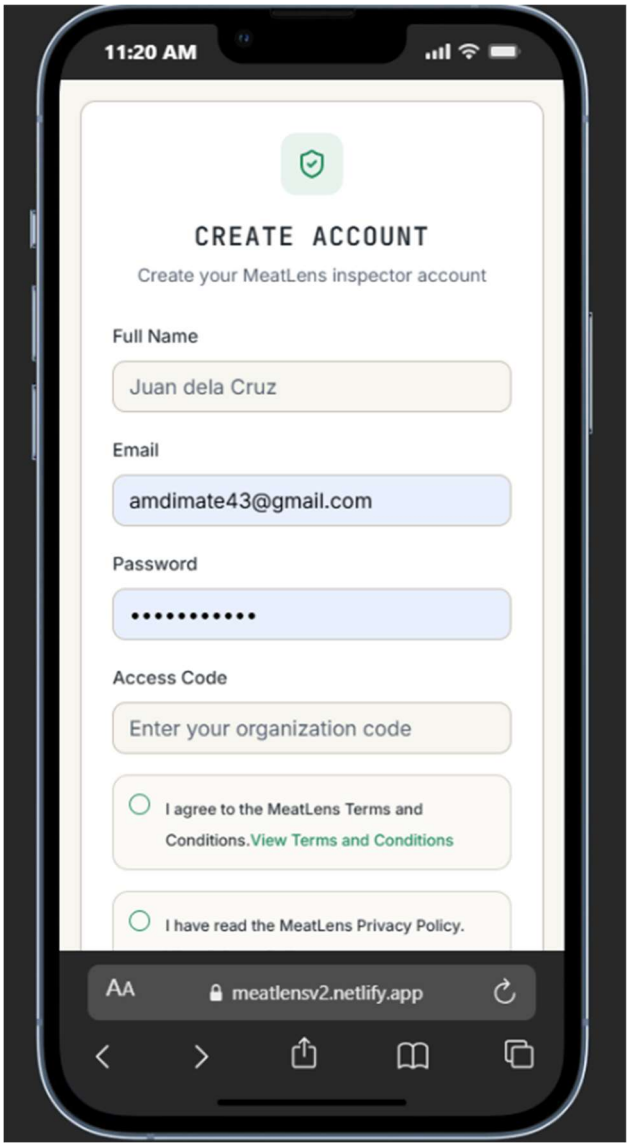
3.2 Sign In (Existing User)

- Enter email and password on the sign-in screen.
- Inspector users are redirected to inspection flow; admins to admin dashboard.
- Offline credential cache allows limited sign-in when network is unavailable.

Sign in page (Sign in button)



Sign up page (Create inspector account)

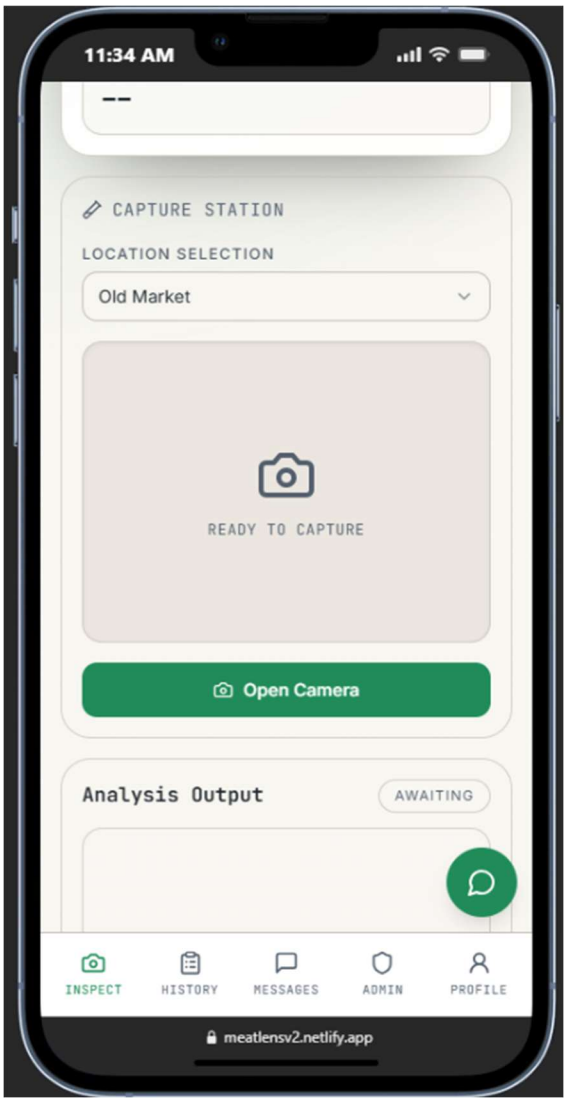


3.3 Password Recovery and Reset

- Use `Forgot password?` and type in the email to receive a reset link.
- Once a link is received via email, reset password from there.
- Reset page validates token and applies new password through recovery endpoint.
- Signed-in users can also change password directly from Profile page.

4. Inspection Workflow (User Guide)

Inspection flow is centered around capture, AI analysis, confidence interpretation, and saved records.



4.1 Capture Station

- Select market location before capture.
- Tap `Open Camera`, frame sample inside guide box, then capture.
- Optional debug file upload is restricted to unlocked admin developer options.

4.2 Analyze and Interpret Result

- Run `Analyze Sample` to execute local MobileNetV3 ONNX inference.
- Result card returns classification, confidence score, and explanation.
- If confidence is below 80%, app warns and recommends retake before save.

4.3 Save and Offline Behavior

- Online save uploads image and creates inspection record in backend.
- Offline save queues image and metadata in IndexedDB for automatic sync on reconnect.
- OfflineSyncManager uploads pending scans and audit events when network returns.

4.4 Classification Outputs

- Fresh: high confidence quality state.
- Not Fresh or Acceptable: moderate quality decline requiring timely action.
- Warning: low-confidence or degraded sample requiring caution and possible retake.
- Spoiled: strong spoilage indicators; not suitable for consumption.

5. History, Reporting, and Traceability

The History module supports inspection auditability through filtering, search, and export workflows.

5.1 Filtering and Search

- Filter by classification (`All`, `Fresh`, `Not Fresh`, `Acceptable`, `Warning`, `Spoiled`).
- Search by meat type, location, classification, or inspection ID.
- Date selector focuses records on a specific day for operator-level review.

5.2 Pagination and Metrics

- History list paginates records to maintain mobile readability.
- Summary metrics include total inspections, average confidence, and freshness/spoilage rates.

5.3 Export

- Daily detailed report can be exported as printable HTML and saved as PDF through browser print dialog.
- Each report includes classification, confidence, location, and inspector notes.

6. Messaging and Collaboration

User chat supports direct communication between inspectors and administrators.

6.1 Messaging

- To message an admin, click the message tab, and click the administrator.
- Type in a message and click send.

6.2 Contact Rules

- Inspectors can chat with admin contacts only.
- Admins can chat with inspector contacts only.
- Self-chat and admin-to-admin chat are blocked by service validation.

6.2 Conversation Behavior

- Thread view polls contacts and messages at fixed intervals for freshness.
- Message length is constrained to 1-2000 trimmed characters.
- Newest activity is surfaced in contact preview list.

7. Profile and Preferences

Profile settings allow user information updates and presentation preferences.

- Update name, email, avatar, and password from Profile page.
- Copy inspector code for operational references.
- Toggle light/dark theme and inspect detail complexity preference.
- Open embedded Terms and Privacy dialogs directly from profile section.
- Sign out using confirmation dialog for safe session closure.

8. Admin Dashboard Operations

Admin users access a multi-tab dashboard for governance, analytics, and control.

8.1 Core Tabs

- Overview: KPI cards, trend charts, role distribution, and class analytics.
- Users: create, edit, and delete inspector/admin accounts.
- Inspections: review and remove records with preview support.
- Access Codes: generate, enable/disable, copy, and delete registration codes.
- Markets: maintain selectable market location list for field inspectors.
- Reports: export PDF summary, CSV detail, and JSON snapshot by date range.
- Logs: review decrypted audit events with actor and source metadata.
- Developer Options: unlock debug toggles and queue maintenance tools.

8.2 Safety and Auditability

- Destructive actions require confirmation dialogs before execution.
- Admin account mutations emit audit events for traceability.
- Developer options require admin role plus password-derived unlock token.

9. Technical Architecture Reference

MeatLens follows a React + Express + Supabase architecture with client-side AI inference.

9.1 Frontend Layer

- React 18 + TypeScript application built with Vite.
- UI components use Tailwind and shadcn patterns.
- React Query caches server data; offline behavior uses IndexedDB queues.
- Routes include public auth pages, protected inspector pages, and admin dashboard.

9.2 Backend Layer

- Express + TypeScript API server with service-controller-route pattern.
- CORS allow-list supports wildcard origin matching.
- Auth token validation resolves actor context and role for protected operations.
- Health endpoint: `/api/analysis/health`.

9.3 AI Inference Model Flow

- Inference is client-side only; server `/analyze` endpoint is intentionally retired (HTTP 410).
- Image is cropped to guide box, resized to model input (default 224x224), then normalized.
- Model metadata controls preprocessing mode and output label ordering.
- Softmax probabilities map to classification and confidence-aware warning behavior.

10. API Reference

The backend exposes REST endpoints under `/api`. Authenticated requests must include `Authorization: Bearer <access_token>`.

Route Group	Endpoints
/api/auth	POST /sign-in, POST /sign-up, POST /sign-out, POST /reset-password, PATCH /users/:id/email, PATCH /users/:id/password, POST /recovery/password
/api/analysis	POST /analyze (retired; returns 410), GET /health
/api/inspections	GET /, GET /stats, GET /:id, POST /, DELETE /:id
/api/profiles	GET /stats, GET /, GET /:id, PUT /:id, admin user create/update/delete endpoints
/api/access-codes	POST /validate, GET /, POST /, DELETE /:id, PATCH /:id/toggle
/api/market-locations	GET /, POST /, DELETE /:id
/api/upload	POST /inspection-image
/api/stats	GET /landing-page
/api/chat	POST / (SSE response stream)
/api/audit-logs	GET / (admin), POST / (batch event ingestion)
/api/developer-options	POST /unlock, POST /verify
/api/user-chat	GET /contacts, GET /messages/:counterpartyId, POST /messages

11. Data Model and Security Controls

Supabase PostgreSQL stores inspections, profiles, roles, access codes, market locations, audit logs, and chat messages.

11.1 Core Tables

- `profiles`: user metadata, inspector code, dark mode, detailed-result preference.
- `user_roles`: role mapping with admin/user semantics.
- `inspections`: classification record with confidence, explanation, notes, and capture metadata.
- `access_codes`: controlled registration mechanism with activation and usage limits.
- `market_locations`: admin-managed market selector values.
- `audit_logs`: encrypted append-only system event ledger.
- `user_chat_messages`: constrained admin-user messaging records.

11.2 Security Layers

- Row-level security policies isolate user-owned data and permit admin read scopes where defined.
- Storage policies scope image write/read/delete to user folders and admin oversight rules.
- Audit log payloads are encrypted using AES-256-GCM with key ID tagging.
- Developer options unlock token uses HMAC SHA-256 signature and TTL enforcement.
- Chat assistant endpoint applies strict scope policy for meat-safety and app-usage topics only.

11.3 Offline Integrity

- Client submission IDs prevent duplicate inspection writes during retry/sync.
- Pending scans and audit events are durable in IndexedDB and removed only after successful sync.

12. Deployment and Environment Configuration

Recommended production topology uses Netlify for the frontend and Render for the backend, both sourced from the same monorepo.

12.1 Required Environment Variables

Variable	Service	Purpose
VITE_API_BASE_URL	Frontend	Base URL for backend API (example: https://your-backend.onrender.com/api)
SUPABASE_URL	Backend	Supabase project URL
SUPABASE_SERVICE_KEY	Backend	Service key for privileged backend database/storage operations
ALLOWED_ORIGINS	Backend	Comma-separated CORS allow list, supports wildcard subdomains
UPLOAD_DIR	Backend	Temporary upload staging directory
AUDIT_LOG_KEY	Backend	AES-256-GCM key used to encrypt audit log payloads
AUDIT_LOG_KEY_ID	Backend	Identifier for the active audit key (default: v1)
DEVELOPER_OPTIONS_PASSWORD	Backend	Admin-only unlock password for developer options
DEVELOPER_OPTIONS_TOKEN_SECRET	Backend	HMAC secret for developer options unlock token
DEVELOPER_OPTIONS_TOKEN_TTL_SECONDS	Backend	Developer options token

		lifetime in seconds
GROQ_API_KEY	Backend	API key for the chat assistant stream endpoint

12.2 Build and Run Commands

- `npm install` installs dependencies for both workspaces.
- `npm run dev` starts frontend and backend concurrently.
- `npm run dev:frontend` starts Vite client (default port 8080).
- `npm run dev:backend` starts Express API (default port 3001).
- `npm run build` builds backend and frontend for deployment.
- `npm start` runs the backend production server from compiled output.

12.3 Deployment Notes

- Netlify uses `netlify.toml` with base directory `frontend/` and SPA rewrite rules.
- Render uses `render.yaml` and health check `GET /api/analysis/health`.
- Set backend CORS (`ALLOWED_ORIGINS`) to include Netlify production and preview domains.
- Use a keep-awake health ping schedule if your Render tier idles inactive services.

13. Troubleshooting

Use this quick diagnostic table before escalating issues to development support.

Issue	Likely Cause	Recommended Action
Cannot sign in	Invalid credentials or expired session token	Re-enter credentials, use Forgot Password, and confirm backend `/api/analysis/health` is reachable.
Sign-up fails	Missing or invalid access code, password too short	Use an active access code issued by admin and ensure password is at least 6 characters.
Camera will not open	Browser permission denied or device camera busy	Allow camera permission in browser settings and close apps using the camera.
Inspection save failed while online	Upload or API request failed	Retry save, verify internet, and inspect backend logs for upload or auth errors.
Offline scan not syncing	Device remains offline or sync failed mid-queue	Reconnect to network, keep app open, and allow OfflineSyncManager to drain queue.
Admin dashboard data missing	User lacks admin role or API auth	Sign out/in with admin account

	expired	and confirm `user_roles` includes `admin`.
Audit logs unavailable	Audit key env var missing or role restrictions	Set `AUDIT_LOG_KEY` and ensure admin role for read access via `/api/audit-logs`.
CORS errors in browser	Frontend origin not included in backend allow list	Update `ALLOWED_ORIGINS` and redeploy backend.

14. Maintenance Checklist

Use this checklist for routine technical upkeep.

- Verify health endpoint and CORS policy after each deployment.
- Confirm model artifact files are present before frontend build.
- Rotate `AUDIT_LOG_KEY` using controlled key-id migration process.
- Review admin audit logs for anomalous account or deletion events.
- Test offline capture and sync path after major frontend updates.
- Validate access code lifecycle and market location accuracy monthly.